📝 Lab Session 4
ADVANCED SYNCHRONIZATION & CONCURRENCY PATTERNS

Part2 – Monitors

The limitations of semaphores

- Wait and signal are scattered across multiple threads/processes, and they must match.
- It is not always easy to understand what is happening.
- Their use must be correct in all threads/processes.
- A single thread can cause an entire collection of threads to fail (e.g., forgetting to call signal).

Monitors

A **monitor** is a **high-level synchronization mechanism** used in concurrent programming to safely control access to shared data by multiple threads.

Think of a monitor as a **"box"** that contains:

1. **Shared data**
2. **A lock (mutex)**
3. **Condition variables** for waiting and signaling
4. **Methods** that are **automatically synchronized**

Only **one thread at a time** can execute code inside the monitor.

Monitors simplify concurrent access by embedding synchronization directly into the object structure.

✔ Key properties of a monitor

1. Mutual Exclusion

Only **one thread** can run inside the monitor at a time. This prevents **race conditions.**

2. Condition Variables

A monitor allows threads to:

- await() → release the lock and wait

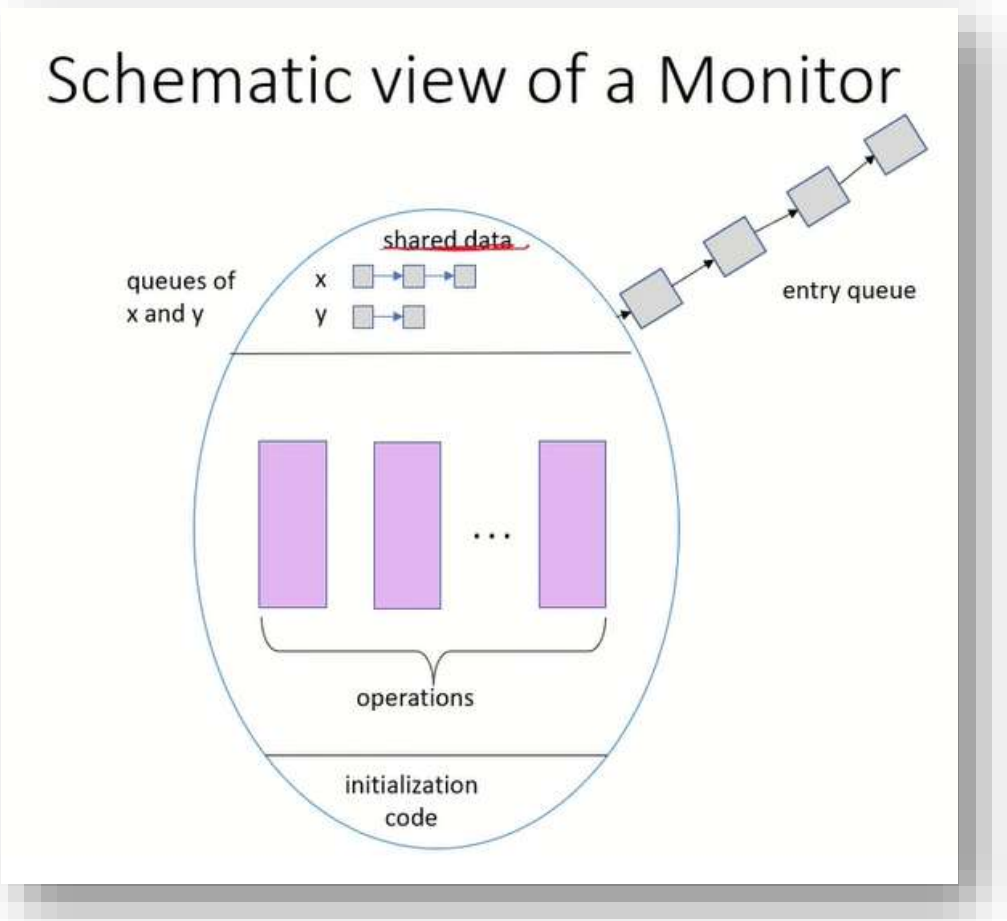
- signal() → wake one waiting thread
- Signalall() → wake all waiting threads

3. Encapsulation

Combines shared data with the procedures that operate on it. Protecting the data from external interference -→ The shared data stays ***inside* the monitor,** only accessed through **synchronized methods**.

Recap:

- Local variables are accessible only through a procedure of the monitor.
- A monitor procedure manipulates only the monitor's variables.
- A thread enters the monitor by invoking one of its procedures.
- At any given time, only one thread can execute inside the monitor, but multiple threads/processes can be waiting in the monitor.
- A monitor alone ensures mutual exclusion of its procedures; there is no need to program it explicitly.
- Shared data protection is ensured by placing the data inside the monitor.
- The monitor locks the shared data when a thread enters it.
- Thread synchronization is carried out using condition variables, which represent conditions that a thread may wait for before executing inside the monitor.



THREAD WAKE-UP STRATEGY AFTER EXECUTING A SIGNAL

- If a thread/process **P** executes **x.signal()** and releases a thread/process **Q**
- There are then **two threads/processes that want to execute**, **P** and **Q**, which is not allowed.

Two possible strategies:

1. **Signal and wait: P** waits until **Q** exits the monitor.
    - **Motivation:** If **Q** is not woken immediately, the condition for which it was awakened might no longer be valid.
2. **Signal and continue: P** continues its execution, and **Q** may wait until **P** exits the monitor.
    - **Motivation:** To avoid a context switch.

Demo code:

```
package LAB4;

/**
 * Five hungry people, anxiously waiting for their turn to take soup
 */

import java.util.concurrent.locks.*;

class HungryPerson extends Thread {

    private int personID;
    private static Lock cookerLid = new ReentrantLock();
    private static int servings = 11;
    private static Condition soupTaken = cookerLid.newCondition();

    public HungryPerson(int personID) {
        this.personID = personID;
    }

    public void run() {
        while (servings > 0) {
            cookerLid.lock();
            try {
                while ((personID != servings % 5) && servings > 0) {
// check if it's not your turn
                    System.out.format("Person %d checked... then put
the lid back.\n", personID);
                    soupTaken.await();
                }
                if (servings > 0) {
                    servings--; // it's your turn - take some soup!
```

```
                    System.out.format("Person %d took some soup!
Servings left: %d\n", personID, servings);
                    soupTaken.signalAll();
                }
            } catch (Exception e) {
                e.printStackTrace();
            } finally {
                cookerLid.unlock();
            }
        }
    }
}

public class ConditionVariableDemo {
    public static void main(String args[]) {
        for (int i=0; i<5; i++)
            new HungryPerson(i).start();
    }
}
```

Key lines explanation:

This program simulates **5 hungry people** (threads).
There are **11 servings of soup**, but **each person is allowed to take soup only when it is their turn**. Whose turn is it?

Examples:

- servings = 11 → 11 % 5 = 1 → **Person 1** may take soup
- servings = 10 → 10 % 5 = 0 → **Person 0** may take soup
- servings = 9 → 9 % 5 = 4 → **Person 4** may take soup
- servings = 8 → 8 % 5 = 3→ **Person 3** may take soup
- servings = 7 → 7 % 5 = 2 → **Person 2** may take soup
- servings =5→ 6 % 5 = 1 → **Person 1** may take soup
- servings = 5 → 5 % 5 = 0 → **Person 0** may take soup
- servings = 4 → 4 % 5 = 4 → **Person 4** may take soup
- servings = 3 → 3 % 5 = 3 → **Person 3** may take soup
- servings = 2 → 2 % 5 = 2 → **Person 2** may take soup
- servings = 1 → 1 % 5 = 1→ **Person 1** may take soup

So each person must **wait for their turn**, using a **Condition Variable**.

Components of the Monitor in the Code

1 Reminder: What is a Monitor

In Java (or concurrent programming):

- A **monitor** is a structure that combines a shared resource + a locking mechanism + conditions for thread synchronization.
- It ensures that **only one thread at a time** can access the shared variables.
- Threads can **wait for a condition** and **wake up when it changes**.

2 Components of the Monitor in Your Code

In your code, the monitor elements are:

```
private static int servings = 11;          // shared variable
private static Lock cookerLid = new ReentrantLock();  // lock
private static Condition soupTaken = cookerLid.newCondition(); // condition variable
```

private static int servings = 11; **// shared variable**
private static Lock cookerLid = new ReentrantLock(); **// lock**
private static Condition soupTaken = cookerLid.newCondition(); **// condition variable**

- servings → the **shared resource**: the number of soup servings.
- cookerLid → the **monitor lock**: ensures mutual exclusion, meaning only one thread can modify servings at a time.
- soupTaken → the **monitor condition**: allows threads to wait for their turn and be woken up when another thread takes a serving.

3 Where the Monitor Acts

In the run() method:

```
cookerLid.lock();  // start of monitor
try {
    while ((personID != servings % 5) && servings > 0) {
        soupTaken.await();  // thread waits for its condition
    }
    if (servings > 0) {
        servings--;  // protected access to the shared resource
        soupTaken.signalAll(); // wake up other threads
    }
} finally {
    cookerLid.unlock();  // end of monitor
}
```

- The block cookerLid.lock() … cookerLid.unlock() acts as the **entry and exit of the monitor**.
- Inside:
  - Only the current thread can access servings.
  - If it is not its turn (personID != servings % 5):
    - You call await():
    - releases the lock
    - puts the thread to sleep
    - waits until someone calls signalAll()
  - When a thread takes a serving, it calls **signalAll**() to wake up waiting threads.

- **Always release the lock**

```
} finally {
   cookerLid.unlock();
}
```

Even if an exception occurs, the lock is released. This ensures no **deadlock** happens.

4 Summary

The monitor in your code is **not a special class**; it is implemented using ReentrantLock and Condition.

- **Shared resource:** servings
- **Mutual exclusion:** cookerLid.lock()/unlock()
- **Thread synchronization:** soupTaken.await()/signalAll()